

A Comparative Study of Task Offloading Approaches in the Edge-Cloud Paradigm

Gurman Kaur

Department of Computer Science & Technology
Algoma University
Brampton, ON, Canada
gurmankaur01@algonau.ca

Faria Khandaker

Department of Computer Science & Technology
Algoma University
Brampton, ON, Canada
faria.khandaker@algonau.ca

Abstract—Cloud computing has been instrumental in expanding the reach and capabilities of computing, storage, and networking infrastructure to applications. However, its centralized architecture leads to increased latency, bandwidth consumption, and service interruptions, making it unsuitable for latency-sensitive applications such as autonomous vehicles, smart cities, and industrial automation. Edge computing mitigates these challenges by relocating computation closer to data sources, such as IoT devices and edge servers, thereby reducing response time and enhancing Quality of Service (QoS). Instead of replacing cloud computing, edge computing works together with it to create a hybrid edge cloud system that combines fast local processing with the cloud’s powerful storage and computing capacity. However, this integration imposes challenges in efficient task offloading and scheduling and resource management. This paper presents a comprehensive analysis of several existing task-offloading strategies of edge-cloud environments in terms of several performance metrics such as task failure, network delay, processing time and edge utilization. The fuzzy logic-based algorithm is found out to be the most effective task offloading algorithm among all considered algorithms as it considers a number of network and computational resources while making task offloading decisions.

Index Terms—Edge computing, cloud computing, task offloading, latency, Quality of Service (QoS), hybrid architecture.

I. INTRODUCTION

Cloud computing has significantly changed how data is processed, stored, and managed by offering scalable, flexible, and on-demand access to computing resources. It allows organizations to handle large-scale applications without investing heavily in physical infrastructure. However, the centralized structure of cloud computing often introduces challenges such as high network latency, excessive bandwidth consumption, and potential service disruptions. These limitations become especially critical in real-time and delay-sensitive applications, such as autonomous driving, smart cities, healthcare monitoring, and industrial automation, where a fast response time is essential [1].

To address these issues, edge computing has emerged as a promising solution. Instead of sending all data to centralized cloud servers, edge computing processes data closer to the source on local edge servers, gateways, or even directly on IoT devices. This reduces the time it takes for data to travel across the network, leading to lower latency, better bandwidth utilization, and improved Quality of Service (QoS)

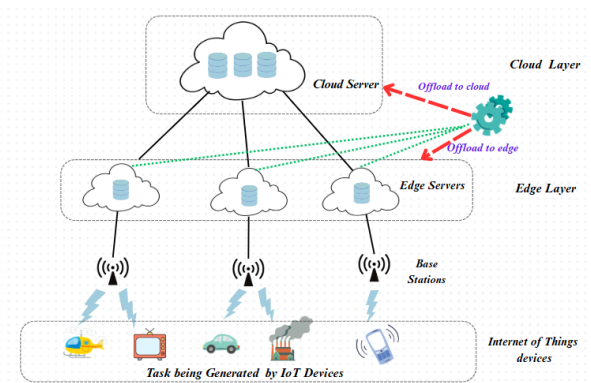


Fig. 1: Edge-Cloud Architecture

[2]. Edge computing is especially useful for applications that require immediate processing or operate in environments with limited or unreliable connectivity. Rather than serving as a replacement, edge computing works alongside cloud computing within a hybrid architecture commonly referred to as edge-cloud computing, as illustrated in Figure 1. This architecture typically consists of three layers: the IoT layer, which generates tasks; the edge layer; and the cloud layer, which handles data processing and storage. Positioned between the edge and cloud layers is often a decision-making entity known as an edge orchestrator, which is responsible for task offloading and resource management [3]. This combination brings together the real-time responsiveness of the edge with the high computational capacity and storage capabilities of the cloud. While this integration offers many advantages, it also introduces new challenges, particularly in how to efficiently offload and schedule tasks between the edge and the cloud. Managing resources, balancing workloads, minimizing delays, and ensuring scalability remain major concerns in task offloading and scheduling [4].

Our paper focuses on analyzing and evaluating several existing task offloading strategies. We have evaluated and compared five task offloading strategies in terms of task failure, network delay, processing time, and edge server utilization. By analyzing how these task offloading strategies perform under different conditions, we have identified the

most effective task offloading approaches that improve task offloading decision-making, reduce task failures, and enhance the overall performance of edge-cloud systems.

The remainder of this paper is organized as follows. Section II provides an overview of the edge cloud architecture. Section III elaborates on the task offloading approaches considered for our analysis. Section IV consists of the details of the performance evaluation results of the task offloading approaches and Section V consists of the final discussions, and concludes the paper.

II. LITERATURE REVIEW

The rise of the Internet of Things(IoT) has led to a huge increase in data generation and demand for real-time data processing. The traditional centralized cloud computing often cannot meet the low-latency needs of many IoT applications. As a result, computing models like edge and fog computing were introduced that brought processing closer to the devices, reducing delays and improving performance. El-Sayed et al. [5] explain how edge computing, IoT, and cloud systems can work together in a model called “Edge of Things”. This model helps reduce network traffic and supports fast, real-time responses. The paper highlights how edge devices can perform tasks such as decision-making before sending data to the cloud. Yosefpour et al. [3] provide a broad overview of edge-related computing models such as fog, mist, cloudlets, and mobile edge computing across multiple levels between the IoT devices and the cloud. The authors describe how fog computing reduces delays, saves bandwidth, and scales better for large systems. The paper also covers how these systems can work with technologies such as 5G and software-defined networking (SDN), making them useful for smart cities, connected cars, and more. Cao et al. [4] focus on how edge computing supports cyber-physical systems (CPS), which combine physical devices and digital systems. The authors emphasize on the need for smart scheduling systems that can adjust to changing network conditions and device availability. Sonmez et al. [6] suggest using fuzzy logic to manage workloads in edge systems. Their task offloading method considers task length, CPU utilization, and network delay. Task offloading decisions in edge-cloud systems are also studied by Almutairi and Aldossary [1]. They present a novel hybrid offloading model that uses resource availability, delay tolerance, and energy consumption to determine the optimal offloading destination. Flores and Srirama [7] explore offloading in mobile devices by introducing an adaptive offloading framework that combines fuzzy logic with evidence-based learning to optimize code execution in mobile cloud applications. The system consists of a fuzzy decision engine on the mobile device and a learning mechanism that continuously refines offloading decisions based on historical execution traces.

III. TASK OFFLOADING ALGORITHMS

In our paper, we analyze several task offloading approaches and provide a detailed performance analysis to identify the

effective approaches for optimizing resource allocation, reducing latency, and improving system efficiency in edge-cloud environments. The in-depth analysis of the offloading approaches are provided below.

We have analyzed and evaluated five different task-offloading algorithms, which are: Network-Based, Utilization-Based, Hybrid-Based, Fuzzy competitor and Fuzzy Logic-Based algorithms. We have evaluated the algorithms based on some key performance metrics, such as failed tasks, network delay, processing time and edge server utilization. These metrics determine the effectiveness of the task offloading algorithms for optimizing task allocation, ensuring efficient resource utilization, minimizing delays, and enhancing overall system performance in edge-cloud environments.

1) *Network-based, Utilization-based, and Hybrid:* The Network-Based approach uses network bandwidth as a key parameter when determining where to offload tasks. If the WAN bandwidth exceeds 6 Mbps, the task is offloaded to the cloud; otherwise, it is offloaded to the edge.

The Utilization-Based approach assesses CPU usage of the edge servers. If CPU utilization is below 80%, the task is directed to the edge; if it exceeds 80%, the task is offloaded to the cloud.

The Hybrid approach integrates both network-based and utilization-based approaches. Specifically, if the WAN bandwidth exceeds 6 Mbps and the average VM utilization exceeds 80%, the task is offloaded to the cloud; otherwise, it is handled by the edge server. The threshold values of 6 Mbps for bandwidth and 80% for CPU utilization in the above approaches are empirical [6].

2) *Fuzzy-based:* Fuzzy logic is employed to determine whether a task should be offloaded to an edge server or the cloud, based on multiple parameters. This decision-making process occurs in two stages as follows.

In the first stage, the system selects the most suitable edge server by evaluating factors such as network delay, edge server utilization, and remote edge utilization. It uses 27 predefined fuzzy rules to calculate membership values for both nearest edge servers and remote edge servers. An accumulation function, specifically, the center of gravity method is then used to compute the final output value for this task offloading approach. If this value is less than 50, the task is offloaded to the nearest edge server; otherwise, it is directed to a remote edge server.

In the second stage, task-specific characteristics such as task size, delay sensitivity, available bandwidth, and virtual machine (VM) utilization are analyzed. Using 81 additional fuzzy rules on the values of input parameters, the system computes updated membership values for cloud and edge options. If the resulting value is below 50, the task is offloaded to the edge; otherwise, it is sent to the cloud. [6]

3) *Fuzzy Competitor:* This approach utilizes fuzzy logic to make task offloading decisions based on input parameters such as WAN bandwidth, CPU speed, video execution requirements, and task data size [6]. It applies 81 predefined fuzzy rules on the values of input parameters to compute

membership values for both cloud and edge servers. An accumulation function is then applied to derive the final output value. If the output value is less than 50, the task is offloaded to the edge server; otherwise, it is directed to a cloud server.

IV. PERFORMANCE EVALUATION

This section presents an in-depth performance analysis of the considered task-offloading algorithms. The task offloading algorithms are implemented in EdgeCloudSim [6], an edge-cloud simulation framework. For our simulation, the computational capacity of the edge layer is represented by 6 edge servers. The number of IoT devices is varied between 200 and 1,000. We consider four application types: augmented reality, infotainment, healthcare, and computation-intensive applications. Our simulation architecture incorporates an edge orchestrator, which functions as the decision-making entity for task offloading. The performance analysis of the task offloading algorithms for varying numbers of IoT devices is presented as follows.

1) *Failed tasks*: The average task failure percentage is one of the most crucial performance criteria for task offloading. Tasks fail due to four main reasons which are mobility, lack of VM capacity on the edge servers, and lack of MAN and WAN capacity.

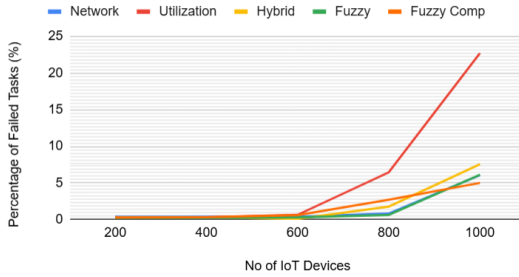


Fig. 2: Percentage of Failed Tasks

The average task failure percentage for varying number of IoT devices is illustrated in Figure 2. All approaches perform similarly under light system load. However, as the load increases, the fuzzy-based approach consistently outperforms the others. This is because it effectively balances both computational and networking resources, dynamically adapting its offloading decisions based on the real-time status of the resources. The Fuzzy Competitor approach also achieves low task failure percentage under heavy loads but it performs worse than the fuzzy-based method under lighter loads, primarily because it does not take the status of Metropolitan Area Network (MAN) resources into account. This oversight leads to increased network losses on the MAN, even when the system is lightly loaded. The Utilization-Based strategy yields the worst performance. It operates with a fixed threshold (80% edge server utilization), offloading tasks to the cloud once edge resources become congested. Over time, this leads to WAN congestion, resulting in a high rate of task failures on the network. The Network-Based approach shows relatively low task failure percentages overall, but still experiences

significant task drops due to limitations in MAN capacity. Since it only considers WAN bandwidth and offloads tasks to edge servers once the WAN becomes congested, it fails to effectively manage resource distribution, particularly under high-load conditions.

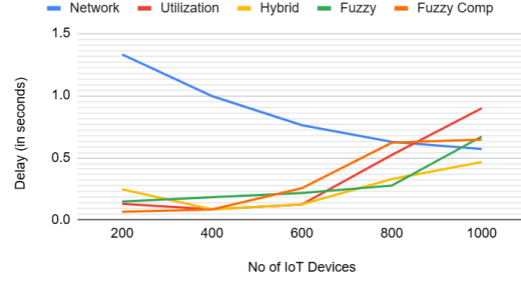


Fig. 3: Network Delay

2) *Network Delay*: Network delay refers to the time required for a task to travel from one point in a network to another, i.e., to travel from IoT devices to either edge or cloud. The network delay observed across various algorithms is illustrated in Figure 3.

When the system load is low, the Network-Based algorithm exhibits the worst performance as it considers that WAN bandwidth is inversely proportional to WAN delay. Under light load conditions, WAN delay is minimal, resulting in higher available bandwidth. Since this available bandwidth exceeds the 6 Mbps threshold, the algorithm offloads more tasks to the cloud, which in turn increases the overall network delay. In contrast, other algorithms demonstrate comparable performance under low loads (a smaller number of IoT devices), with relatively lower network delay. However, as the system load increases, the Utilization-Based approach performs the worst. This is because, once edge servers' CPU resources become congested (80% edge server utilization), the algorithm shifts task offloading to the cloud, thereby increasing network delay. The Hybrid approach, which considers both network and CPU utilization parameters, performs better than the individual network-based and utilization-based strategies, as it makes more balanced offloading decisions. The Fuzzy Competitor tends to favor offloading to edge servers, which helps reduce network delay under higher loads. However, the Fuzzy-based algorithm consistently delivers the best performance, maintaining a gradual and controlled increase in network delay.

3) *Processing Time*: Processing time refers to the time required to execute a task and is a key factor affecting overall task completion time. The processing time required by the different algorithms are shown in Figure 4. When the number of IoT devices is smaller, the Network-Based approach yields the best performance as it tends to offload tasks to the cloud when network delay is minimal. The cloud offers significantly higher processing power than edge servers, resulting in shorter processing times. In contrast, the Fuzzy, Utilization-Based, and Hybrid approaches primarily offload tasks to edge servers. Since edge servers have limited computational

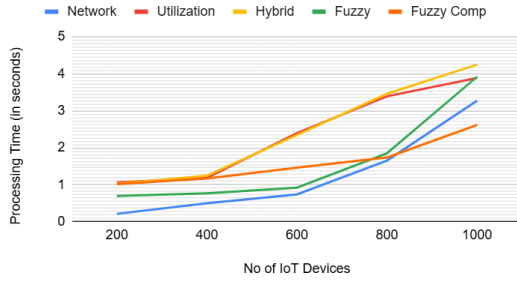


Fig. 4: Processing Time

capacity compared to the cloud, they yield higher processing time, performance degrades as the number of IoT devices, i.e., the volume of tasks, increases. This increased workload leads to higher processing time. The Fuzzy Competitor performs better than utilization-based and hybrid under heavier loads, as it takes a broader range of computational parameters into account. By intelligently distributing tasks between edge and cloud resources, it achieves a more balanced load and lower processing time.

4) *Edge Server Utilization*: Edge utilization refers to the average CPU usage of VMs running on edge servers. The edge utilization across various algorithms is illustrated in Figure 5.

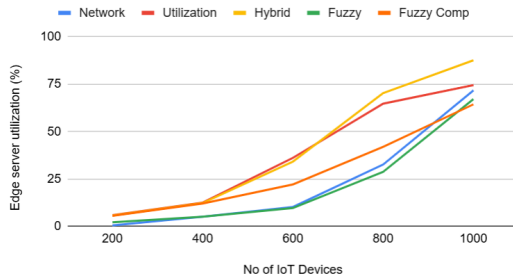


Fig. 5: Edge Server Utilization

The Network-Based approach shows lower edge utilization under light system loads. With fewer IoT devices, network delay is minimal, WAN bandwidth is high, and since the available bandwidth exceeds the 6 Mbps threshold, tasks are primarily offloaded to the cloud. The Fuzzy-based algorithm exhibits similar behavior under low load conditions, as it evaluates both network and computational parameters when making offloading decisions. The Fuzzy Competitor demonstrates higher edge utilization than both the Fuzzy-based and Network-Based approaches at low loads. However, its utilization increases more gradually as the load rises, indicating more balanced task distribution. The Utilization-Based and Hybrid algorithms prioritize offloading to edge servers and only redirect tasks to the cloud once edge CPU utilization exceeds the 80% threshold, resulting in consistently higher edge utilization levels across all load conditions. Overall, the Fuzzy-based algorithm strikes the best balance by effectively utilizing both edge and cloud resources. Its adaptive, multi-parameter decision-making enables it to maintain high efficiency across different system loads, making it the most

effective strategy across various performance metrics.

V. CONCLUSION

Despite advancements in edge computing, efficient task offloading remains a critical challenge for optimizing performance. The choice of an appropriate offloading mechanism has a significant impact on computational efficiency, resource utilization, and latency reduction. This paper provides a comparative analysis of various task offloading strategies, assessing their ability to balance workload distribution and improve overall system performance. Among the evaluated approaches, the Fuzzy-based algorithm consistently demonstrates superior performance across multiple metrics. Its dynamic and adaptive decision-making process considering both computational and network conditions, enables more effective resource utilization and reduced task failure. The Fuzzy Competitor also produces efficient results by intelligently distributing tasks based on a variety of computational factors. In contrast, the Utilization-Based approach performs the worst in most scenarios. Its reliance on a fixed 80% edge server utilization threshold led to congestion at the edge layer, resulting in higher task failure rates. The Hybrid algorithm followed a similar trend, as it also incorporates this threshold in its decision-making logic. The Network-Based approach shows relatively better performance than utilization-based and hybrid approaches, particularly under higher loads, due to its ability to shift tasks to the edge as WAN congestion increases. However, its limited consideration of computational parameters hindered its overall effectiveness. In conclusion, adaptive strategies, particularly those based on fuzzy logic offer the most promising solution among all considered algorithms for optimizing task offloading in edge computing environments.

REFERENCES

- [1] J. Almutairi and M. Aldossary, "A novel approach for iot tasks offloading in edge-cloud environments," *Journal of cloud computing*, vol. 10, no. 1, p. 28, 2021.
- [2] A. Asghari and M. K. Sohrabi, "Server placement in mobile cloud computing: A comprehensive survey for edge computing, fog computing and cloudlet," *Computer Science Review*, vol. 51, p. 100616, 2024.
- [3] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, and J. P. Jue, "All one needs to know about fog computing and related edge computing paradigms: A complete survey," *Journal of Systems Architecture*, vol. 98, p. 289–330, Sep. 2019. [Online]. Available: <http://dx.doi.org/10.1016/j.sysarc.2019.02.009>
- [4] K. Cao, S. Hu, Y. Shi, A. W. Colombo, S. Karnouskos, and X. Li, "A survey on edge and edge-cloud computing assisted cyber-physical systems," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 11, pp. 7806–7819, 2021.
- [5] H. El-Sayed, S. Sankar, M. Prasad, D. Puthal, A. Gupta, M. Mohanty, and C.-T. Lin, "Edge of things: The big picture on the integration of edge, iot and the cloud in a distributed computing environment," *IEEE Access*, vol. 6, pp. 1706–1717, 2018.
- [6] C. Sonmez, A. Ozgovde, and C. Ersoy, "Fuzzy workload orchestration for edge computing," *IEEE Transactions on Network and Service Management*, vol. 16, no. 2, pp. 769–782, 2019.
- [7] H. Flores and S. Srirama, "Adaptive code offloading for mobile cloud applications: Exploiting fuzzy sets and evidence-based learning," 06 2013, pp. 9–16.